

Go Auto Demo-1

Loyalty-Driven Dealer Performance

Service Cost Prediction (Regression
+ Anomaly Detection)



Presented by : Team Alpaca, February 10, 2026

Objectives

- ★ **Assess data quality and understand loyalty program distribution**
- ★ **Explore preliminary patterns in customer behaviour toward loyalty vs. non-loyalty.**
- ★ **Conduct initial dealer-level performance comparison**
- ★ **Set foundation for clustering analysis in Demo 2**



Company Overview

GoAuto

Go Auto, founded in the early 2000's, boasts a network of more than 70 dealerships across Canada and the USA. The company is recognised for its integrated automotive services and award-winning management practices. It is a:

- Multi-location automotive dealership group
- Thousands of service transactions yearly
- Revenue driven by service + customer retention
- Focus on loyalty, fairness, and customer trust

Problem Statement

**Loyalty-Driven Dealer Performance
(Clustering + Comparative Analysis)**

How does loyalty program participation impact dealership performance, and what customer segments drive different loyalty behaviors across GO Auto's seven Edmonton locations?

Problem Statement:

Specific Objectives:

- 1.Loyalty Impact Assessment
- 2.Customer Segmentation
- 3.Dealer Benchmarking
- 4.Geographic Analysis:

Business Alignment:

This analysis supports GO Auto's strategic priority of improving customer retention beyond five years by showing which dealers best use loyalty programs and which customer segments offer the highest retention value.

Dataset

Data Source: GO Auto Services Customer Database (2020–2024)
Records: 242,818 service transactions
Dealerships: 7 locations across Edmonton

Key Variables:

Variable	Type	Description
<code>service_date</code>	String/Date	Date of service visit
<code>sale_date</code>	String/Date	Original vehicle purchase date
<code>customer_pay</code>	Integer	Amount paid by customer () <i> 'warranty_pay' Integer</i> <i> Amount covered by warranty(</i>
<code>year</code>	Integer	Vehicle manufacturing year
<code>make</code>	String	Vehicle manufacturer
<code>model</code>	String	Vehicle model
<code>vin</code>	Integer	Vehicle identification number (customer proxy)
<code>loyalty_card</code>	String	Loyalty program participation (Yes/No)
<code>cost</code>	String	Total service cost
<code>appointment</code>	Boolean	Service was scheduled (True/False)
<code>mileage</code>	Integer	Vehicle odometer reading
<code>dealer_name</code>	String	Dealership location
<code>distance</code>	Float	Distance customer traveled to dealership (km)

Dataset

	service_date	sale_date	customer_pay	warranty_pay	year	make	model	vin	loyalty_card	cost	appointment	mileage	dealer_name	distance
0	7/29/2024 0:00	Apr-21	1	0	2021	TOYO	RAV4HY	3395527356	Yes	200-300	True	39076	Toyota on the Trail	9.0
1	7/30/2024 0:00	Dec-23	1	1	2024	TOYO	RAV4HY	5296888873	Yes	0-100	True	39556	Toyota on the Trail	12.0
2	7/2/2024 0:00	Mar-20	1	1	2020	LAND	RR	9475224721	No	1300-1400	True	66001	Jaguar Land Rover Edmonton	10.8
3	7/3/2024 0:00	Jul-23	1	1	2018	NISS	PATH	3446880696	No	500-600	True	82454	Go Nissan South	15.3
4	10/30/2022 0:00	May-11	1	0	2008	FORD	FUSN	5577514554	No	0-100	True	5	Kentwood Ford	4.8
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
242813	9/18/2023 0:00	Feb-23	1	0	2020	FORD	F150	1813236058	Yes	1400-1500	True	104768	Kentwood Ford	10.2
242814	4/17/2020 0:00	Aug-17	1	0	2017	FORD	F150	6891633666	No	300-400	False	99848	Kentwood Ford	4.0
242815	2/12/2021 0:00	May-15	1	0	2015	FORD	F150	1516333102	No	300-400	False	110669	Kentwood Ford	24.4
242816	1/20/2021 0:00	Jan-19	1	0	2018	FORD	F150	5245369000	No	200-300	False	66496	Kentwood Ford	5.7
242817	9/23/2024 0:00	Oct-14	1	0	2014	FORD	F150	6675358575	No	0-100	False	296883	Kentwood Ford	4.5

EDA

EDA included the following steps:

- Handling Duplicates
- Handling Missing Values
- Data type Conversion
- Feature Engineering
- Outlier Detection
- Correlation Analysis

5.1 Handling Duplicates

```
# check for duplicates
dups = df.duplicated().sum()
print("DUPLICATE RECORDS ANALYSIS")
print(f"duplicate rows: {dups}")
print(f"percentage: {(dups/len(df)*100):.2f}%")

if dups > 0:
    print("found some duplicates - might want to look into this")

... DUPLICATE RECORDS ANALYSIS
duplicate rows: 4
percentage: 0.00%
found some duplicates - might want to look into this
```

Decision tree for duplicates:

- if exact duplicates: Drop them
- if same VIN, same service_date: Flag
- if same VIN, different dates: Keep (legitimate customers)

```
# data types
print("data type counts:")
print(df.dtypes.value_counts())
print("\nnull columns:")
print(df.dtypes)
```

```
# deep dive into missing value patterns
print("MISSING VALUE PATTERN ANALYSIS")

# loyalty_card analysis
print("\n1. LOYALTY_CARD: ")
missing_loyalty = df[df['loyalty_card'].isna()]
print(f"    Only {len(missing_loyalty)} records out of {len(df)}")
print(f"    Current distribution: Yes={len(df[df['loyalty_card']=='Yes'])} ({len(df[df['loyalty_card']=='Yes'])/len(df)*100}%)")
print(f"    Current distribution: No={len(df[df['loyalty_card']=='No'])} ({len(df[df['loyalty_card']=='No'])/len(df)*100}%)")

# cost analysis
print("\n2. COST: ")
missing_cost = df[df['cost'].isna()]
print(f"    Total missing: {len(missing_cost)} records")

print(f"\n    Dealer breakdown for missing cost:")
print(missing_cost['dealer_name'].value_counts())
```


Some examples of EDA(Duplicates)

```
# implement duplicate handling strategy

original_size = len(df)

# STEP 1: Drop exact duplicates only
print("\n1. Removing exact duplicates:")
before_drop = len(df)
df = df.drop_duplicates()
print(f"    Dropped: {before_drop - len(df)} exact duplicate records")
print(f"    Remaining: {len(df)} records")

# STEP 2: Flag same-day multiple services for transparency
print("\n2. Flagging same-day multiple services:")
df['same_day_multi_service'] = df.duplicated(subset=['vin', 'service_date'], keep=False)
multi_service_count = df['same_day_multi_service'].sum()
print(f"    Flagged: {multi_service_count} records ({multi_service_count/len(df)*100:.2f}%)")
print(f"    These represent customers who received multiple services same day")

# STEP 3: Create summary for documentation
print("\n3. Summary:")
print(f"    Original records: {original_size}")
print(f"    After deduplication: {len(df)}")
print(f"    Records dropped: {original_size - len(df)} ({(original_size-len(df))/original_size*100:.4f}%)")
print(f"    Multi-service flags: {multi_service_count}")

# STEP 4: Validate we kept legitimate multiple services
print("\n4. Validation - checking payment pattern diversity:")
multi_service_records = df[df['same_day_multi_service']]
if len(multi_service_records) > 0:
    # Group by VIN+date and check payment diversity
    payment_diversity = multi_service_records.groupby(['vin', 'service_date']).apply(
        lambda x: (x['customer_pay'].nunique() > 1) or (x['warranty_pay'].nunique() > 1),
        include_groups=False
    )
    print(f"    Multi-service with DIFFERENT payment types: {payment_diversity.sum()} (legitimate bundling)")
    print(f"    Multi-service with SAME payment types: {len(payment_diversity) - payment_diversity.sum()} (flagged for review)")
```

1. Removing exact duplicates:
Dropped: 4 exact duplicate records
Remaining: 242814 records
2. Flagging same-day multiple services:
Flagged: 7788 records (3.21%)
These represent customers who received multiple services same day
3. Summary:
Original records: 242818
After deduplication: 242814
Records dropped: 4 (0.0016%)
Multi-service flags: 7788
4. Validation - checking payment pattern diversity:
Multi-service with DIFFERENT payment types: 3088 (legitimate bundling)
Multi-service with SAME payment types: 775 (flagged for review)

We identified 7788 same-day visits across customers, which shows a positive result that there is high engagement with service center.

So only 4 true duplicates were removed.

Missing Values

```
# implement missing value strategy
print("IMPLEMENTING MISSING VALUE STRATEGY")

# track original size
original_size = len(df)

# STRATEGY 1: Drop loyalty_card missing (5 records)
print("\n1. Handling loyalty_card:")
before_loyalty = len(df)
df = df.dropna(subset=['loyalty_card'])
print(f"    Dropped {before_loyalty - len(df)} records with missing loyalty_card")
print(f"    Remaining: {len(df)} records")

# STRATEGY 2: Impute cost based on payment patterns
print("\n2. Handling cost:")
missing_cost_count = df['cost'].isna().sum()
print(f"    Total missing cost: {missing_cost_count}")

# create imputation flag BEFORE imputing
df['cost_imputed'] = df['cost'].isna()

# Step 2a: Warranty services get '0-100' (most warranty work is low/no cost to customer)
warranty_missing = df['cost'].isna() & (df['warranty_pay'] == 1)
df.loc[warranty_missing, 'cost'] = '0-100'
print(f"    Imputed {warranty_missing.sum()} warranty services as '0-100'")

# Step 2b: For remaining (customer-pay), we'll handle after parsing cost
# (need cost_avg calculated first)
still_missing = df['cost'].isna().sum()
print(f"    Remaining missing after warranty imputation: {still_missing}")

print(f"\n3. Summary:")
print(f"    Original records: {original_size}")
print(f"    After cleaning: {len(df)}")
print(f"    Records lost: {original_size - len(df)} ({(original_size-len(df))/original_size*100:.3f}%)")
print(f"    Cost imputation flag created: {df['cost_imputed'].sum()} records flagged")
```

... IMPLEMENTING MISSING VALUE STRATEGY

1. Handling loyalty_card:
Dropped 5 records with missing loyalty_card
Remaining: 242809 records
2. Handling cost:
Total missing cost: 2723
Imputed 2236 warranty services as '0-100'
Remaining missing after warranty imputation: 487
3. Summary:
Original records: 242814
After cleaning: 242809
Records lost: 5 (0.002%)
Cost imputation flag created: 2723 records flagged

Our Approach for Missing Values

1. Impute based on payment type
2. For remaining missing, use dealer specific median
3. Create flag for imputed values

Outlier Detection

```
# outlier detection - domain-specific rules
print("OUTLIER DETECTION")
print("="*60)

# Analyze numeric fields
numeric_fields = ['mileage', 'distance', 'vehicle_age']

for field in numeric_fields:
    print(f"\n{field.upper()}:")

    # IQR method
    Q1 = df[field].quantile(0.25)
    Q3 = df[field].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    iqr_outliers = df[(df[field] < lower) | (df[field] > upper)]
    print(f"    IQR Method: {len(iqr_outliers)} outliers ({len(iqr_outliers)/len(df)*100:.2f}%)")
    print(f"    IQR Bounds: [{lower:.1f}, {upper:.1f}]")

    # Domain rules
    if field == 'mileage':
        extreme = df[df['mileage'] > 500000]
        print(f"    Domain Rule: Mileage > 500,000km: {len(extreme)} records (likely errors)")
    elif field == 'distance':
        extreme = df[df['distance'] > 500]
        print(f"    Domain Rule: Distance > 500km: {len(extreme)} records (suspicious but possible)")
    elif field == 'vehicle_age':
        negative = df[df['vehicle_age'] < 0]
        print(f"    Domain Rule: Negative age: {len(negative)} records (pre-orders, fixed to 0)")
```

Mileage: 6909 Outliers (85 flagged)
Distance: 25822 Outliers(2059 flagged)
Vehicle Age: 2681 Outliers(816 negative ages flagged(pre-orders))

```
# Create outlier flags
print(f"\nCREATING OUTLIER FLAGS:")
df['mileage_outlier_flag'] = (df['mileage'] > 500000).astype(int)
df['distance_outlier_flag'] = (df['distance'] > 500).astype(int)

# Fix negative vehicle ages (2025 vehicles serviced in 2024 = pre-orders)
negative_count = (df['vehicle_age'] < 0).sum()
df.loc[df['vehicle_age'] < 0, 'vehicle_age'] = 0

print(f"    mileage_outlier_flag: {df['mileage_outlier_flag'].sum()} records")
print(f"    distance_outlier_flag: {df['distance_outlier_flag'].sum()} records")
print(f"    Fixed {negative_count} negative vehicle ages (pre-orders)")

print(f"\nOUTLIER STRATEGY:")
print(f"    ✓ FLAG but don't remove (legitimate edge cases exist)")
print(f"    ✓ Use flags for sensitivity analysis in Phase 2")
print(f"    ✓ Document extreme values in Phase 1 Report")
```

```
... OUTLIER DETECTION
=====

MILEAGE:
IQR Method: 6909 outliers (2.85%)
IQR Bounds: [-82488.0, 201440.0]
Domain Rule: Mileage > 500,000km: 85 records (likely errors)

DISTANCE:
IQR Method: 25822 outliers (10.63%)
IQR Bounds: [-13.8, 38.5]
Domain Rule: Distance > 500km: 2059 records (suspicious but possible)

VEHICLE_AGE:
IQR Method: 2681 outliers (1.10%)
IQR Bounds: [-4.0, 12.0]
Domain Rule: Negative age: 816 records (pre-orders, fixed to 0)

CREATING OUTLIER FLAGS:
mileage_outlier_flag: 85 records
distance_outlier_flag: 2059 records
Fixed 816 negative vehicle ages (pre-orders)

OUTLIER STRATEGY:
✓ FLAG but don't remove (legitimate edge cases exist)
✓ Use flags for sensitivity analysis in Phase 2
✓ Document extreme values in Phase 1 Report
```

OUTLIER STRATEGY:

- ✓ FLAG but don't remove (legitimate edge cases exist)
- ✓ Use flags for sensitivity analysis in Phase 2
- ✓ Document extreme values in Phase 1 Report

Correlation Analysis(implementation)

8. Correlation Analysis

```
# correlation analysis with visual matrix
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("CORRELATION ANALYSIS")
print("="*60)

# Select numeric features for correlation
numeric_features = ['customer_pay', 'warranty_pay', 'year', 'mileage', 'distance',
                    'vehicle_age', 'loyalty_binary', 'is_luxury', 'under_warranty']

print(f"\nAnalyzing {len(numeric_features)} numeric features")

# Calculate correlation matrix
corr_matrix = df[numeric_features].corr()

# VISUAL CORRELATION MATRIX
print("\n1. CORRELATION MATRIX VISUALIZATION:")
plt.figure(figsize=(12, 10))

# Create heatmap with annotations
sns.heatmap(corr_matrix,
            annot=True,          # Show correlation values
            fmt='.3f',          # 3 decimal places
            cmap='coolwarm',    # Red (positive) to Blue (negative)
            center=0,           # White at zero correlation
            square=True,        # Square cells
            linewidths=0.5,     # Grid lines
            cbar_kws={'shrink': 0.8})

plt.title('Feature Correlation Matrix\n(Red = Positive, Blue = Negative)',
          fontsize=14, fontweight='bold', pad=20)
plt.xlabel('')
plt.ylabel('')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
```

```
plt.title('Feature Correlation Matrix\n(Red = Positive, Blue = Negative)',
          fontsize=14, fontweight='bold', pad=20)
plt.xlabel('')
plt.ylabel('')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

print("  ✓ Matrix displayed above")

# NUMERIC ANALYSIS
print("\n2. HIGH CORRELATIONS (|r| > 0.5):")
high_corr_found = False
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_val = corr_matrix.iloc[i, j]
        if abs(corr_val) > 0.5:
            print(f"  {corr_matrix.columns[i]} ↔ {corr_matrix.columns[j]}: {corr_val:.3f}")
            high_corr_found = True

if not high_corr_found:
    print("  None found")

print("\n3. LOYALTY CORRELATIONS (sorted by strength):")
loyalty_corrs = corr_matrix['loyalty_binary'].sort_values(ascending=False)
for feat, corr_val in loyalty_corrs.items():
    if feat != 'loyalty_binary' and abs(corr_val) > 0.05:
        print(f"  {feat}: {corr_val:.3f}")

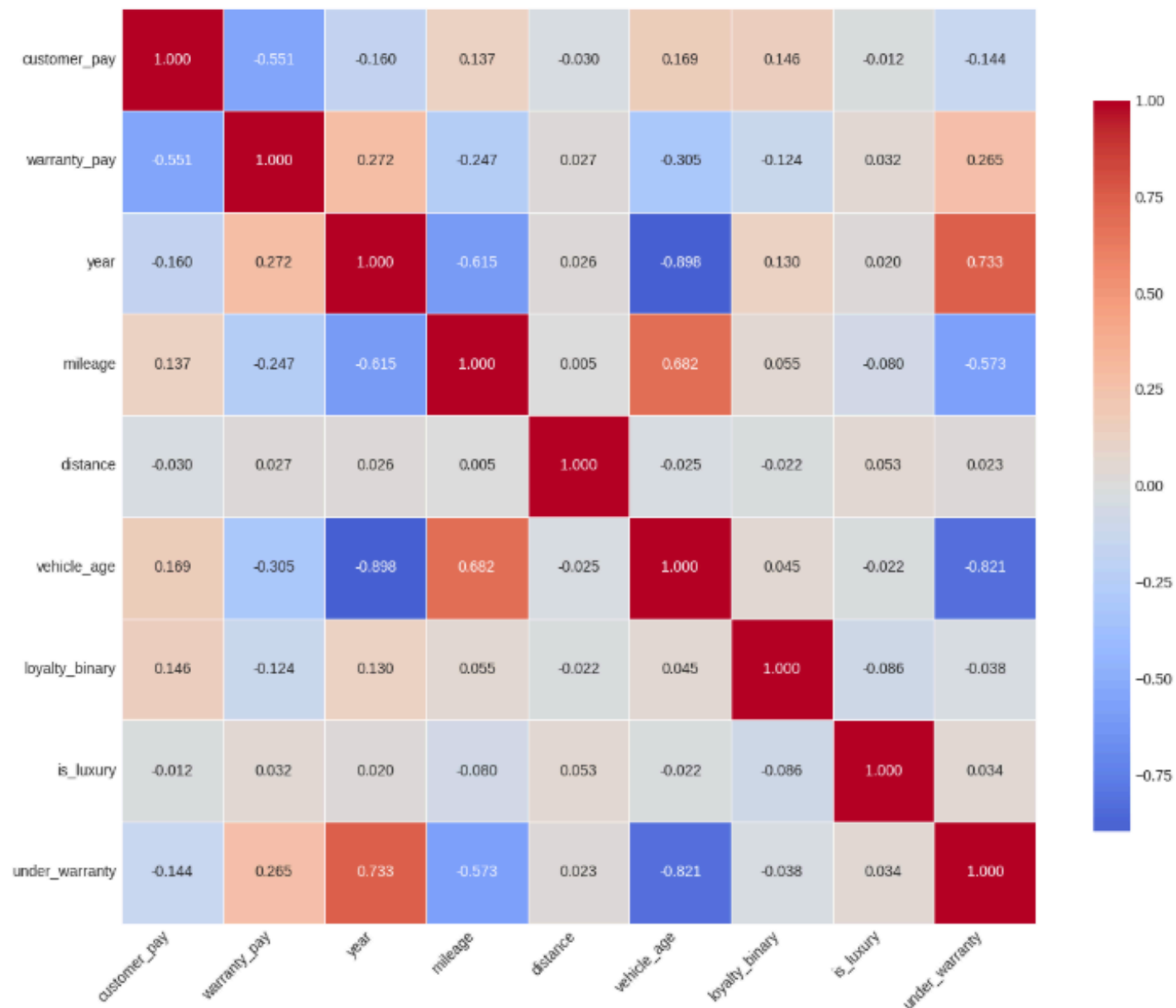
print("\n4. MULTICOLLINEARITY CHECK:")
print("  Features with |r| > 0.7 should not both be used in clustering:")
multicoll_found = False
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_val = corr_matrix.iloc[i, j]
        if abs(corr_val) > 0.7:
            print(f"  ⚠ {corr_matrix.columns[i]} ↔ {corr_matrix.columns[j]}: {corr_val:.3f}")
            multicoll_found = True

if not multicoll_found:
    print("  ✓ No severe multicollinearity detected")
```


Correlation Analysis(Output)

1. CORRELATION MATRIX VISUALIZATION:

Feature Correlation Matrix
(Red = Positive, Blue = Negative)



✓ Matrix displayed above

✓ Matrix displayed above

2. HIGH CORRELATIONS ($|r| > 0.5$):

customer_pay ↔ warranty_pay: -0.551
year ↔ mileage: -0.615
year ↔ vehicle_age: -0.898
year ↔ under_warranty: 0.733
mileage ↔ vehicle_age: 0.682
mileage ↔ under_warranty: -0.573
vehicle_age ↔ under_warranty: -0.821

3. LOYALTY CORRELATIONS (sorted by strength):

customer_pay: 0.146
year: 0.130
mileage: 0.055
is_luxury: -0.086
warranty_pay: -0.124

4. MULTICOLLINEARITY CHECK:

Features with $|r| > 0.7$ should not both be used in clustering:

- ⚠ year ↔ vehicle_age: -0.898
- ⚠ year ↔ under_warranty: 0.733
- ⚠ vehicle_age ↔ under_warranty: -0.821

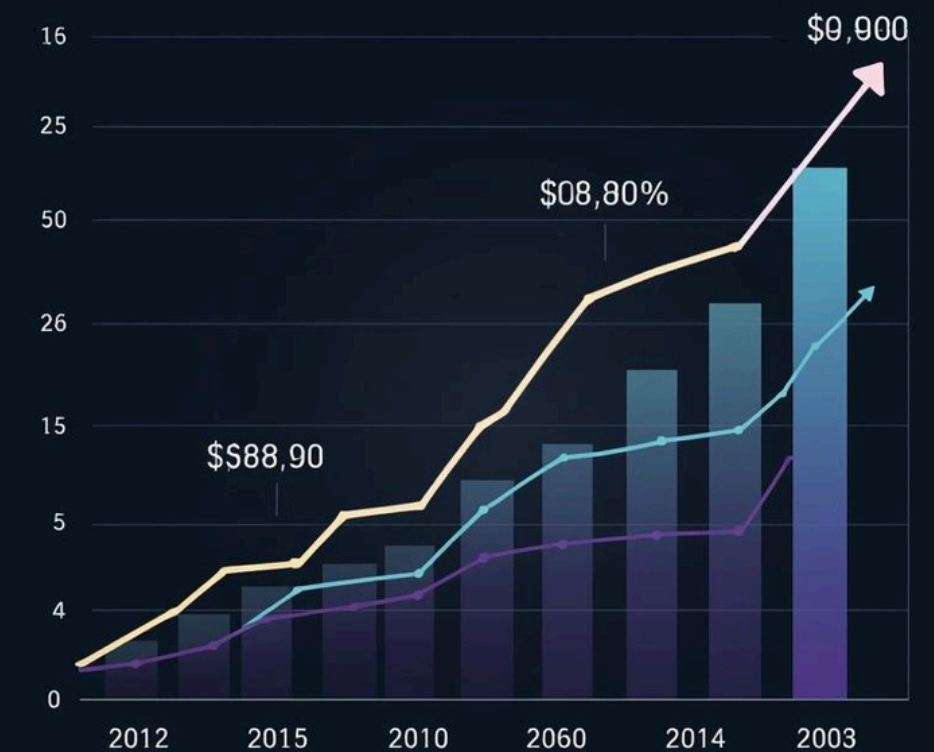
KEY INSIGHTS:

- year & vehicle_age: Highly correlated (expected, redundant)
- vehicle_age & under_warranty: Highly correlated (expected, redundant)
- Loyalty weakly correlated with all features = complex pattern (good for clustering!)
- customer_pay & warranty_pay: Negative correlation (mutually exclusive services)

Exploratory Data Visualizations

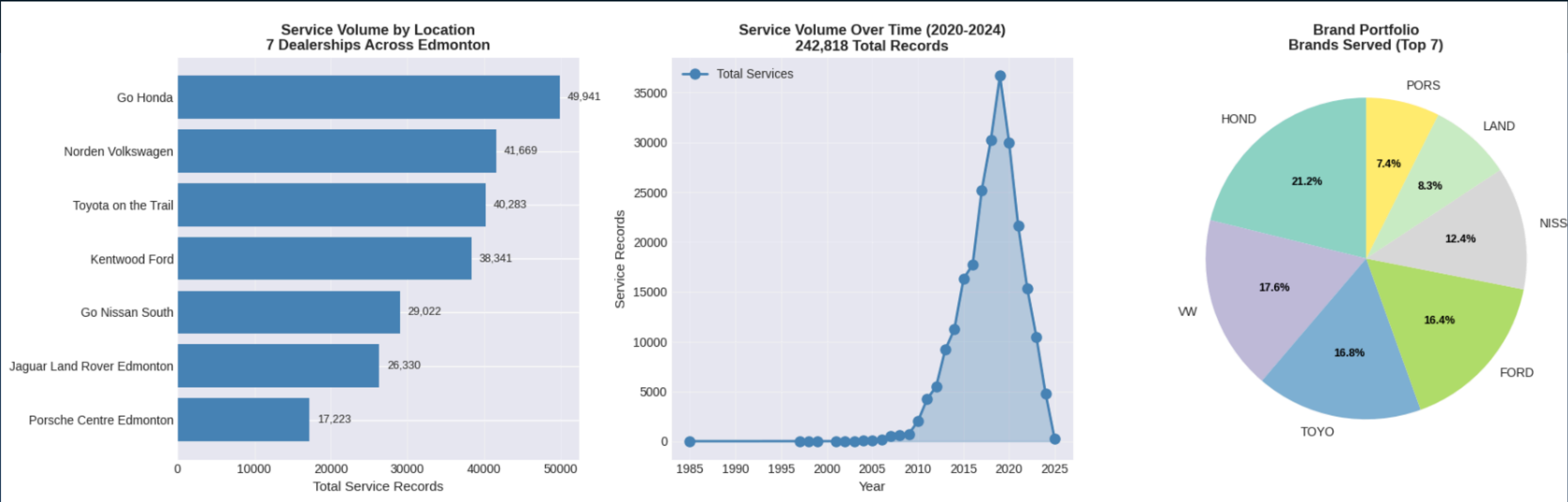
Expected Outcomes:

- Dealer clusters grouped by loyalty adoption & performance
- Customer segments showing loyalty-driven behaviors
- Performance dashboard highlighting effective vs struggling dealerships
- Actionable recommendations for leadership on leveraging loyalty program



Go AUTO at a Glance

Demonstrates comprehensive understanding of GO Auto's scale, structure, and business challenge



Customer Behavior: Loyalty vs Non-Loyalty



82%

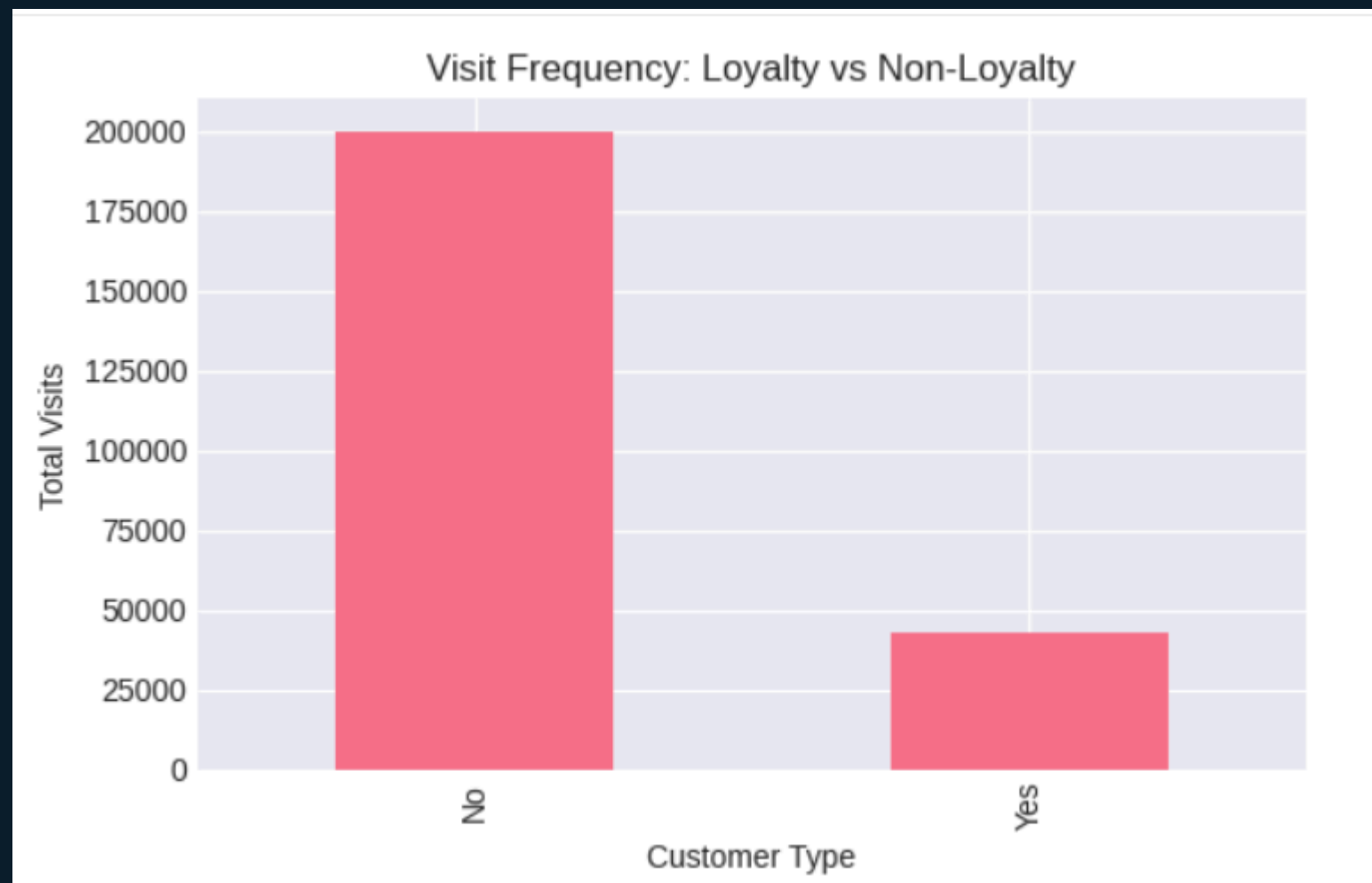
Non-Loyalty Customers

96%

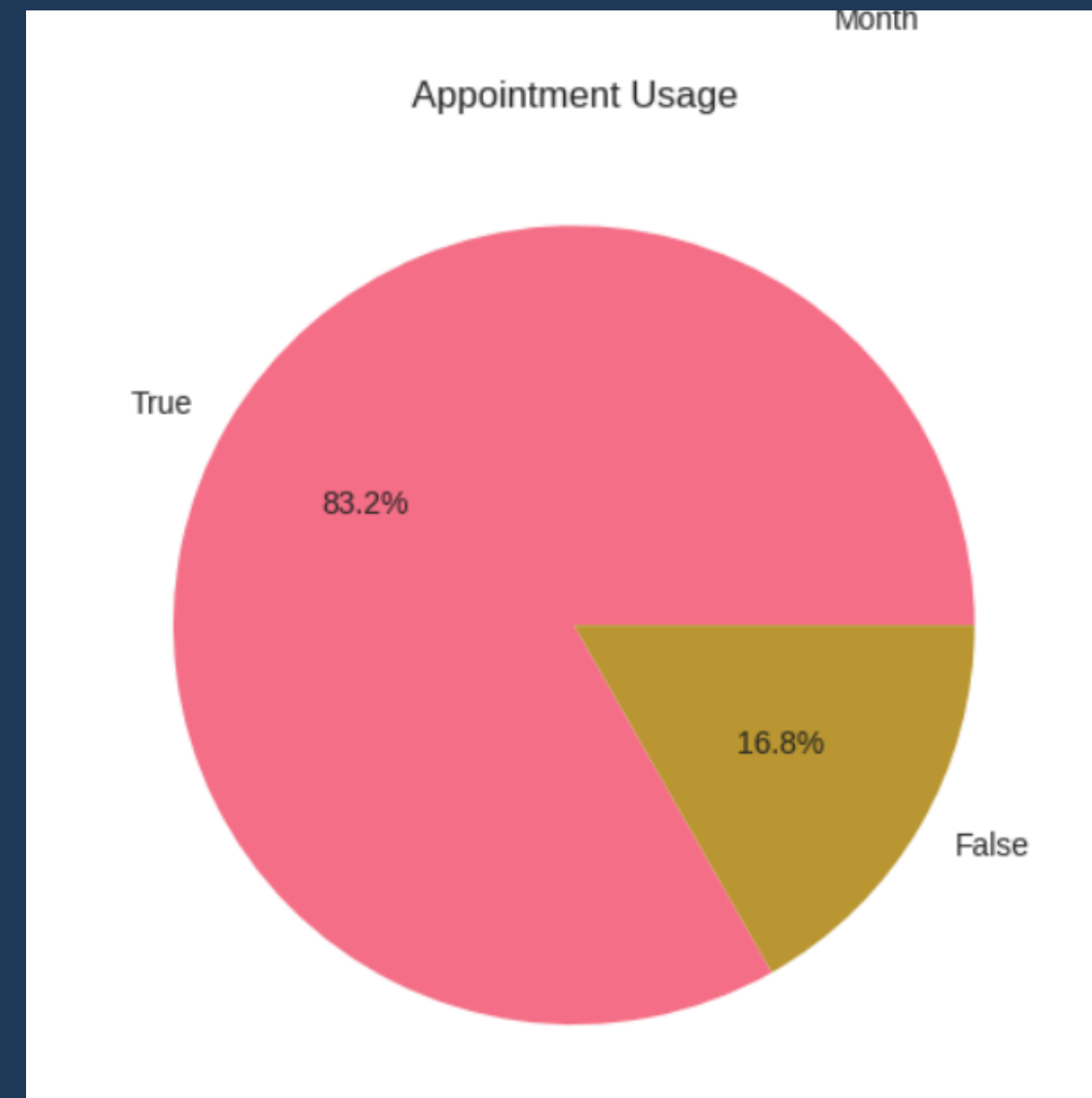
Loyalty Customers

Observation: Loyalty members show higher average service cost and are willing to travel slightly longer distances, indicating stronger engagement and perceived value from the loyalty program.

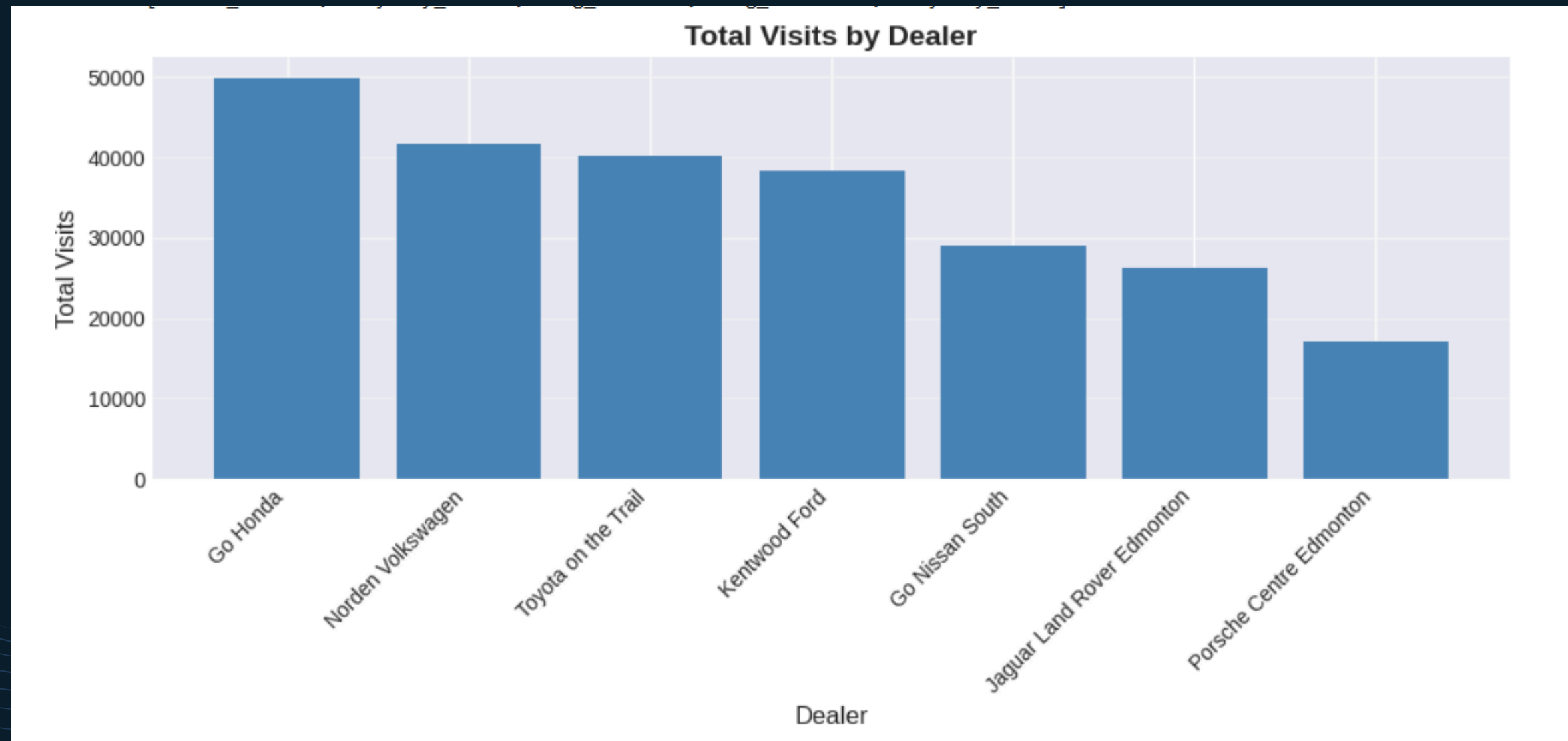
Customer Behavior Analysis:



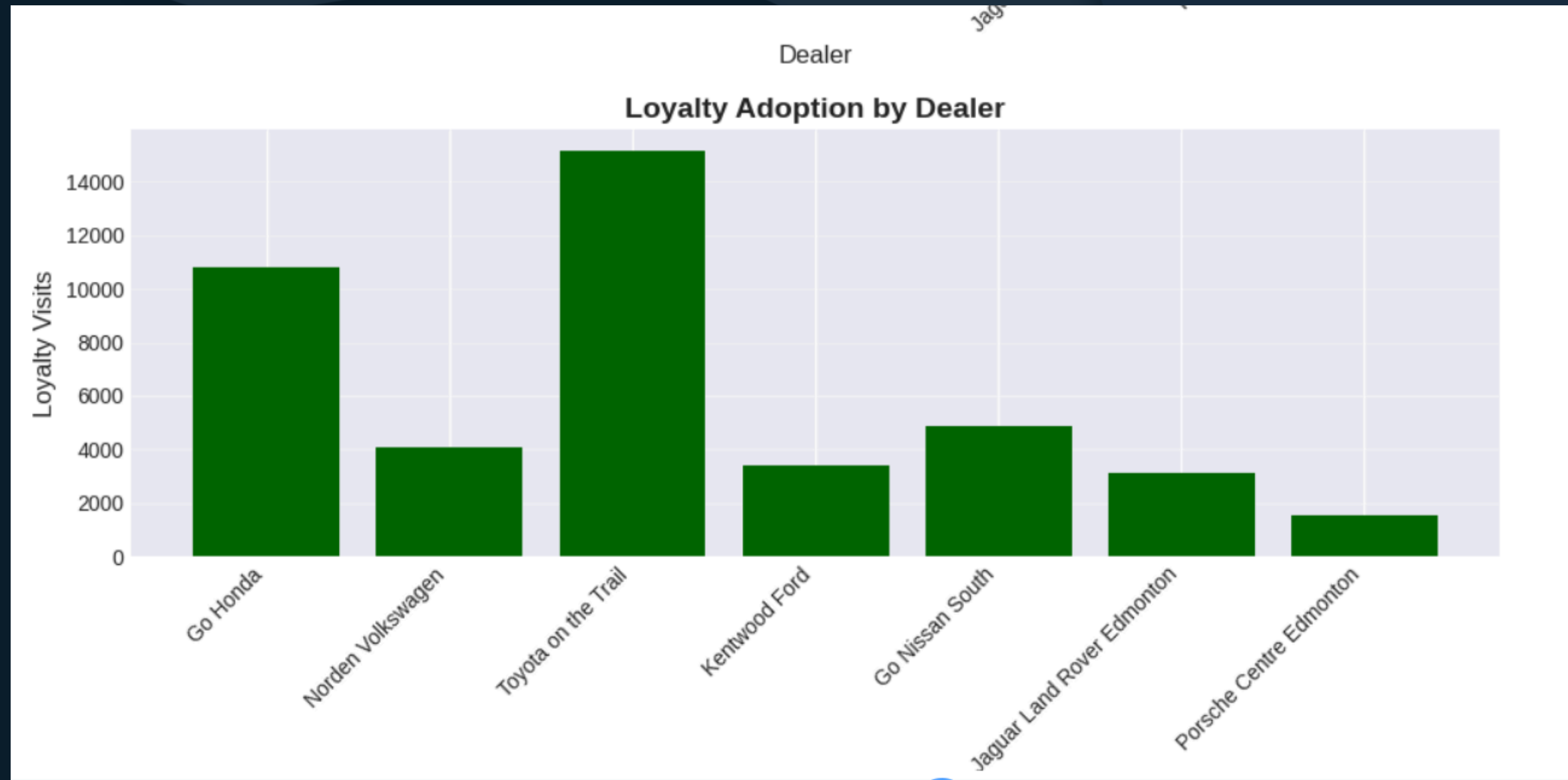
Customer Behavior Analysis:



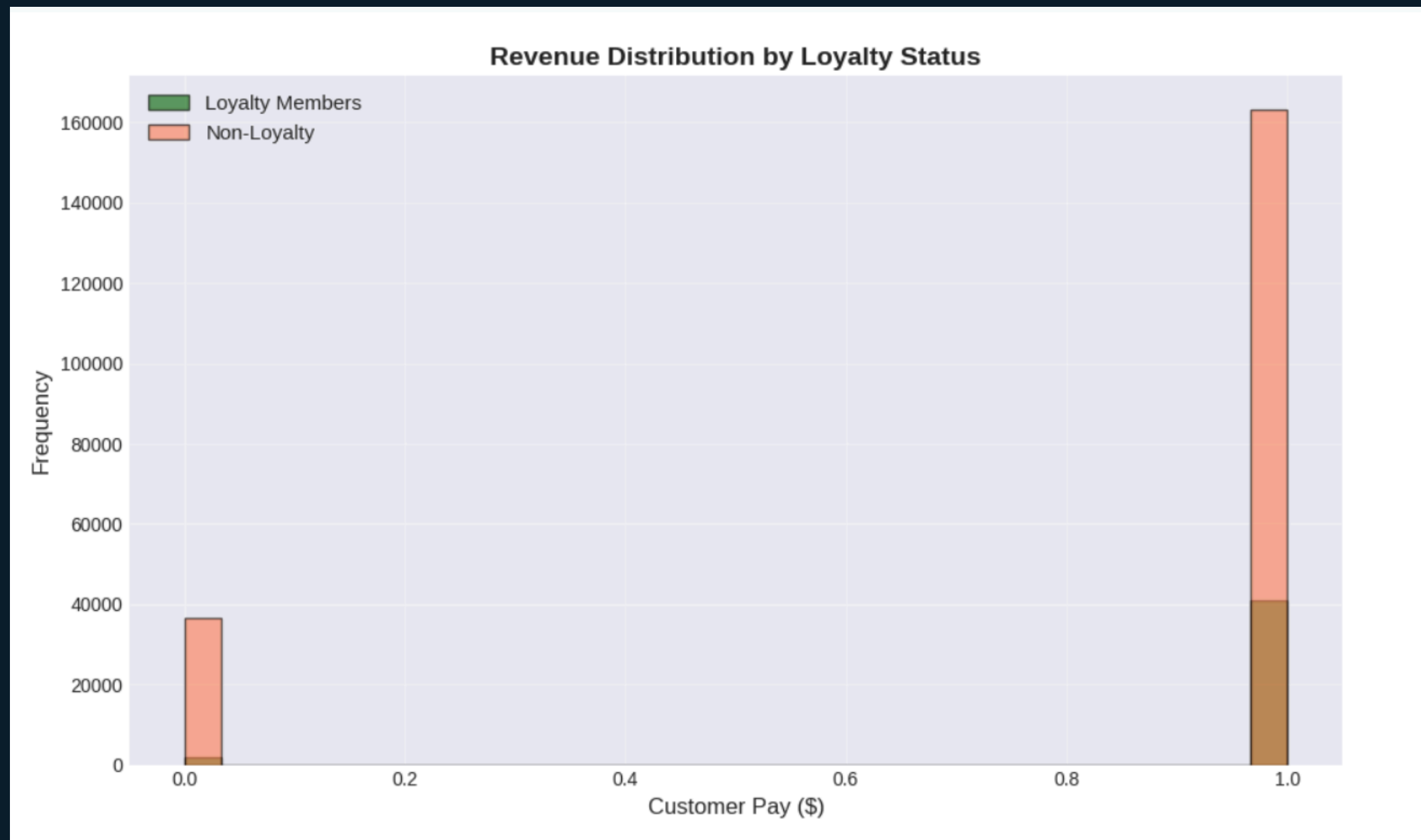
Dealership Performance Overview



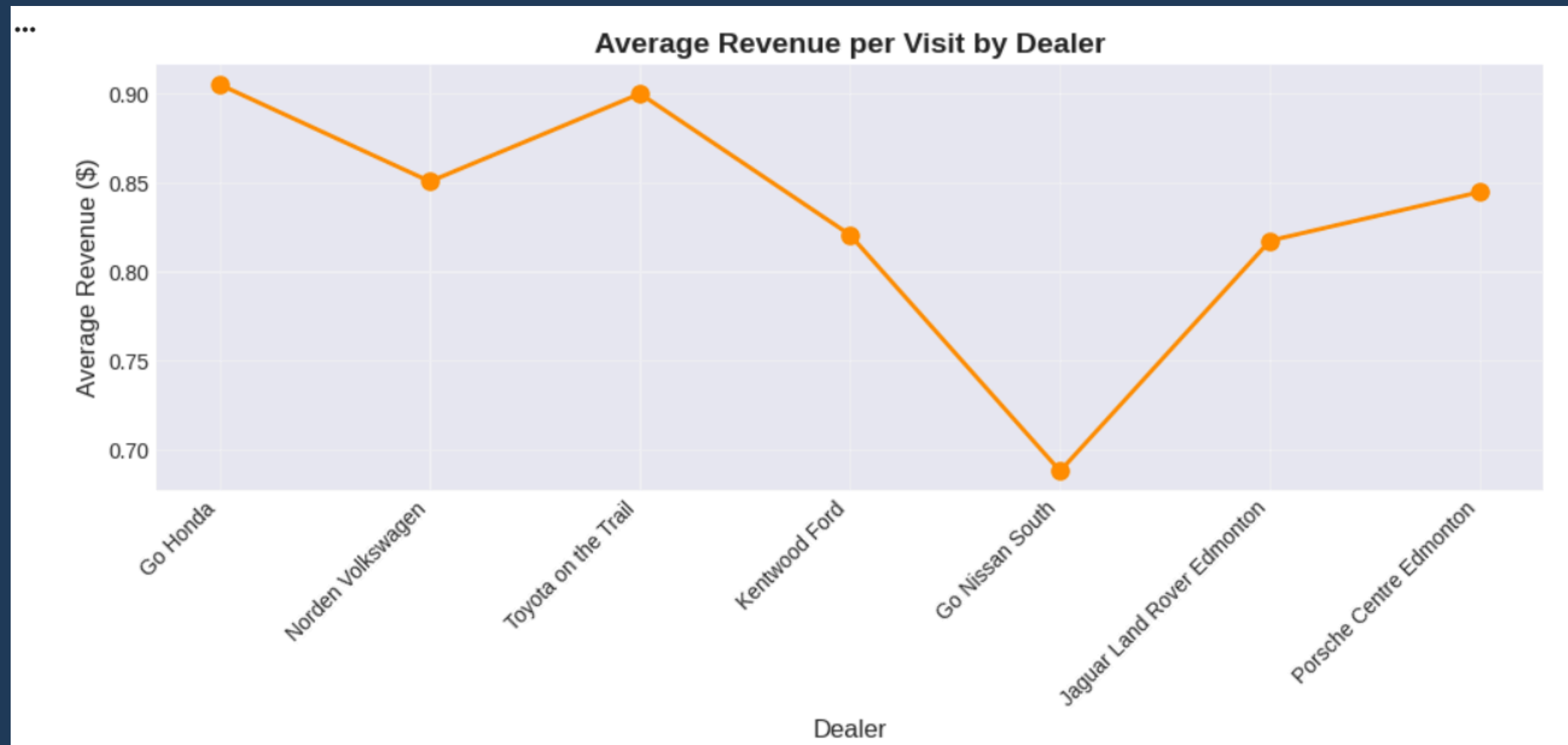
Dealership Performance Overview



Dealership Performance Overview



Dealership Performance Overview



Distance Traveled vs Customer Pay



Key Insights from EDA

Finding 1

Loyalty Adoption Varies Significantly by Dealer

Finding 2

Loyalty members show higher engagement

Finding 3

Loyalty Customers Generate Higher Revenue per Visit

Finding 4

Loyalty members travel farther for service

Next Steps



Customer Segmentation:

Implement clustering algorithm to identify distinct customer groups based on:

- Visit frequency
- Average spending
- Distance traveled
- Loyalty program engagement

Dealer Performance Clustering:

Group dealerships by:

- Loyalty adoption success
- Revenue performance
- Customer retention metrics

Predictive Modeling:

Develop models to:

- Predict customer churn risk
- Forecast loyalty program ROI by dealer
- Identify factors driving customer distance willingness

thank you
Team Alpaca

